In-Memory Compute Macro Using MRAM Crossbar Array

Alexander N. Chin, Sabiq Islam, Harish Eakambaram
Chandra Family Department of Electrical and Computer Engineering
The University of Texas at Austin
Austin, United States of America
achin@utexas.edu sabiqislam@utexas.edu he3837@my.utexas.edu

***Abstract*—Systems continue to scale, latencies and power usage attributed to data transfer between memory and computational blocks have begun to compound and dominate over computational delays. To combat this so called "memory wall", researchers increasingly turned toward in-memory computation as a potential solution. In such a system, computation is executed in the same component where data is stored in memory, hence eliminating the need for the current tiered memory hierarchy (L1-3 cache, SRAM, DRAM, etc). In this work, we focus on the merits of magnetoresistive random access memory (MRAM), and provide an open-source Cadence/Virtuoso based MRAM macro capable of reading/writing, matrix multiplication, and content addressable memory.**

***Index Terms*—MRAM, In-Memory Compute, Memory Wall**

I. Introduction

Modern computing systems are increasingly limited by the memory wall, where the latency and energy required to transfer data between memory and processing units now exceed the cost of computation. In-memory computing (IMC) seeks to overcome this barrier by performing computation within the memory array itself, which reduces data movement and improves overall system efficiency. Several memory technologies have been explored for IMC, including SRAM, ReRAM, and PCM, each offering a distinct balance of volatility, endurance, and analog behavior.

magnetic random-access memory (MRAM), which is built from magnetic tunnel junction (MTJ) devices, provides an attractive platform for IMC because it combines nonvolatile storage, high endurance, and compatibility with standard CMOS processes. MTJs also exhibit predictable resistive behavior that can be used to implement mixed-signal compute operations inside the array. Creating practical MRAM-based compute systems requires both analog circuitry to read and process MTJ outputs and digital logic to manage storage, search, and computation.

In this work, we design the key analog and digital building blocks of an MRAM-based compute macro and validate each subsystem using appropriate industry-standard tools. The analog subsystem, which includes the MTJ crossbar, the read and write paths, the transimpedance amplifier, and the flash ADC, was implemented in Cadence Virtuoso and tested using Spectre simulations. These simulations confirm correct memory behavior, correct multiply-accumulate behavior in the crossbar, and correct generation of quantized voltage levels at the ADC output.

The digital subsystem, which includes a synthesizable content-addressable memory (CAM) engine, a priority encoder, and a finite-state controller, was implemented using a full ASIC design flow consisting of VCS for simulation, Design Vision for synthesis, and Innovus for automated place and route. Functional simulation confirms correct associative search behavior, and the synthesized design meets timing and area constraints, which demonstrates that the digital components operate as intended.

Although unified mixed-signal co-simulation was attempted, full end-to-end integration of the analog and digital subsystems was not completed because of limitations in tool interoperability. Instead, we adopt a modular verification strategy. Each subsystem is fully functional and validated within its correct domain, and the interface specifications between domains are defined to support future mixed-signal integration. This modular approach provides a realistic architectural foundation for an MRAM-based in-memory compute macro and demonstrates how MTJ devices, analog readout circuits, and digital search engines can collectively support future memory-centric computing architectures.

II. Background

Today, the semiconductor industry dominates the global market, with nine of the top ten companies directly involved in semiconductor technology in some form. This dominance is largely driven by the exponential scaling described by Moore's Law. The principle predicts a doubling of transistor density approximately every 2.5 years, which has historically increased computational performance while reducing energy consumption and cost per operation. As a result of this scaling, modern central processing units integrate multiple highly optimized cores, while graphics processing units contain thousands of parallel processing units. Although compute capability has increased rapidly, memory performance has lagged behind.

Current systems face a growing imbalance between processor speed and memory access capability. The delay and energy cost associated with transferring data between memory and compute units now represent a major source of overall system latency and power consumption. This limitation becomes more pronounced in data intensive workloads, particularly when

large parameter sets must be loaded before computation, such as in machine learning inference and training. As researchers explore methods to accelerate data movement and reduce transfer overhead, alternative architectures have been proposed that aim to remove this bottleneck entirely.

One prominent approach is in memory computing, which performs computation directly within the memory array. While this concept can be implemented using conventional CMOS technology and remains compatible with existing fabrication processes, it faces significant challenges in scalability and area efficiency at large system sizes. This work investigates an analog in memory computing approach based on magnetic tunnel junction devices that function as both storage and computational elements, with the goal of improving energy efficiency and throughput compared to traditional Von Neumann architectures.

A. Memory Wall

The memory wall, also known as the Von Neumann bottleneck, describes the growing performance gap between processor execution speed and main memory access latency. While processor clock frequencies and parallelism have increased substantially over time, improvements in dynamic random access memory latency and bandwidth have been comparatively modest. This mismatch forces processors to spend a significant portion of execution time stalled while waiting for data to arrive from memory.

The bottleneck arises from the physical separation of memory and compute units in conventional architectures. Data must traverse off chip or long on chip interconnects, which introduces delay and consumes significant energy per bit transferred. As working data sets grow in size, cache hierarchies become less effective at hiding this latency, and the cost of data movement begins to dominate overall energy consumption. In modern workloads such as graph processing, scientific computing, and machine learning, performance is increasingly limited by memory access rather than arithmetic throughput.

B. MRAM

MRAM is a nonvolatile memory technology that stores information using the magnetic state of nanoscale devices rather than electric charge. The fundamental storage element in MRAM is the MTJ as shown in Fig 1, which consists of two ferromagnetic layers separated by a thin insulating tunnel barrier. One magnetic layer has a fixed magnetization direction, while the other, known as the free layer, can be switched between two stable orientations.

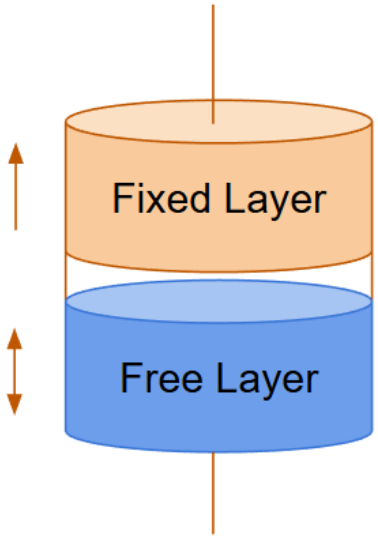

Fig. 1. The following shows the tri-layer structure of a spin transfer torque (STT) MTJ where the top and bottom layers are composed of a ferromagnetic metal separated by a oxide buffer layer in the center. The buffer layer is thin enough to permit electron tunneling forming a so called tunnel barrier. The two metals also act as terminals in device acting like a 2 state resistor.

Data is encoded based on the relative alignment of these layers. When the magnetizations are parallel, the device exhibits a low resistance state. When they are antiparallel, the resistance is higher. These two resistance states represent the digital values. Writing data is typically achieved through spin transfer torque or spin orbit torque, which use polarized current to switch the orientation of the free layer. Reading is performed by applying a small sensing current and measuring the resulting resistance.

MRAM offers several advantages over conventional memory technologies. It provides nonvolatile storage, high endurance, fast read and write speeds, and strong resistance to radiation induced errors. These properties make MTJ based systems attractive not only for standalone memory but also for computing architectures that exploit the analog resistance behavior of MTJs to perform operations such as weighted summation and matrix vector multiplication directly within a memory array. We demonstrate such an architecture in this work as well as analog to digital conversion to allow for seamless integration into a digital system.

The dynamics of the MTJ freelayer can be better understood and simulated via the Landau-Litshitz-Gilbert (1) where m is the unit magnetization vector of the free layer, γ is the gyromagnetic ratio, α is the Gilbert damping constant, and H_{eff} is the effective magnetic field acting on the magnetic layer. The effective field incorporates contributions from the external applied field, anisotropy field, demagnetizing field, and exchange interactions. This formulation captures the precessional motion of the magnetization around the effective field as well as the dissipative torque that drives the system toward its equilibrium orientation. As a result, the Landau–Lifshitz–Gilbert equation provides a physically consistent framework for modeling the

transient and steady-state switching behavior of MTJ free layers during write operations.

$$\frac{\mathrm{d}m}{\mathrm{d}t} = -\gamma m \times H_{eff} + \alpha m \times \frac{\mathrm{d}m}{\mathrm{d}t} \tag{1}$$

In order to read the MTJ, a weak voltage (approximately 0.2 V) is applied so that the sensing operation does not alter the magnetic state. By contrast, a large voltage (on the order of ± 1.0 V) is used to drive a sufficiently large current through the device to switch the orientation of the free layer and thus write a bit. The difference between these two operating regimes defines read and write margins, and maintaining a clear margin is essential for reliable nonvolatile memory operation.

At read level currents, the magnetization remains confined near its initial stable orientation. At write level currents, the torque exerted by the spin polarized current deterministically drives the free layer across the anisotropy energy barrier into the opposite stable state.

When the applied voltage is biased in the intermediate region between the read and write margins, spin transfer torque can induce stochastic switching. In this regime, the applied current is insufficient to guarantee deterministic reversal, but it significantly lowers the effective energy barrier. Thermal fluctuations can then randomly assist the magnetization in crossing this barrier, resulting in probabilistic state transitions. The switching probability depends on pulse amplitude, pulse width, temperature, and device geometry.

This stochastic regime is undesirable for conventional MRAM applications because it leads to read disturb errors and long term retention failures. However, the same effect is advantageous for hardware security and entropy generation, where controlled probabilistic switching can be used to implement true random number generators. For reliable MRAM operation, practical designs ensure that read voltages remain well below this intermediate region and write pulses are sufficiently strong to achieve deterministic switching with low bit error rates.

III. High Level Architecture

The system architecture, shown in Fig. 2, follows a sequential flow of digital control, analog signal generation, in-memory computation, and digital readout. The digital controller supplies the programmed input vectors to the crossbar interface, which distributes the signals as structured digital words. These inputs are converted into analog read and write levels through gated voltage rails that define the read voltage and write voltage applied to each selected line. The resulting analog voltages drive the magnetic tunnel junction (MTJ) crossbar array, which functions as both a nonvolatile memory structure and a compute medium for matrix multiplication inside the array. The analog outputs from the array are digitized by an analog to digital converter (ADC). The digitized values are then forwarded to the CAM controller, which manages data retrieval and prepares the results for subsequent processing.

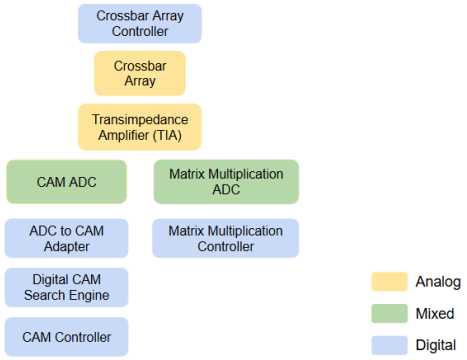

Fig. 2. High-level block diagram of the complete macro architecture. The figure highlights the mixed-signal flow from digital control to analog signal generation and back to digital processing.

A. MTJ Memory Cell

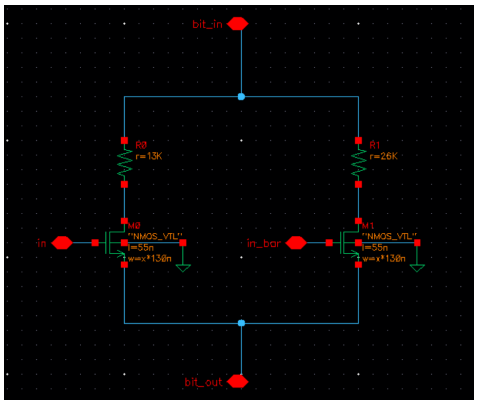

Fig. 3. Schematic of the complementary MTJ memory cell shown in Fig. **??**.d. The MTJs are modeled as fixed resistors for simplicity ($R_{\text{parallel}} = 13\,\text{k}\Omega$ and $R_{\text{anti-parallel}} = 26\,\text{k}\Omega$). As a result, analog write behavior is not implemented at this stage.

The complementary MTJ memory cell consists of two MTJs that store opposite logic states and two NMOS transistors that select which MTJ path is active under control of complementary input signals [1]. This structure provides differential storage and supports both memory and compute functions in the crossbar.

IV. Crossbar Array

The crossbar array uses the MTJ cells described above and supports two modes of operation: memory mode and computational mode. A configurable routing structure switches between the two. In memory mode, the read and write lines access the cells in parallel through dedicated bitlines. In computational mode, the MTJ cells are connected in series within each column, and the summed resistance implements the weighted matrix multiplication.

The multiplication and aggregation method follows [1], and the read and write circuitry follows [2]. In memory mode, the read and write lines apply V_{read} or V_{write} to the MTJ pair and sense the resulting current on shared bitlines. In computational

mode, the MTJ paths are arranged in series, and the output of each column is an analog voltage or current that encodes the sum of the selected cell resistances. This produces an analog output with valid levels from 0 to 3 for the four-cell configuration.

Matrix multiplication in the array is implemented through Ohm's law. For a column containing N cells:

$$I_{\text{out}} = \frac{V_{\text{in}}}{R_1 + R_2 + \cdots + R_N}, \tag{2}$$

$$R_i \in \{R_H, R_L\}, \tag{3}$$

where each R_i represents the selected MTJ branch based on the input and stored weight. The schematic of the MRAM crossbar array is demonstrated in Fig. 5.

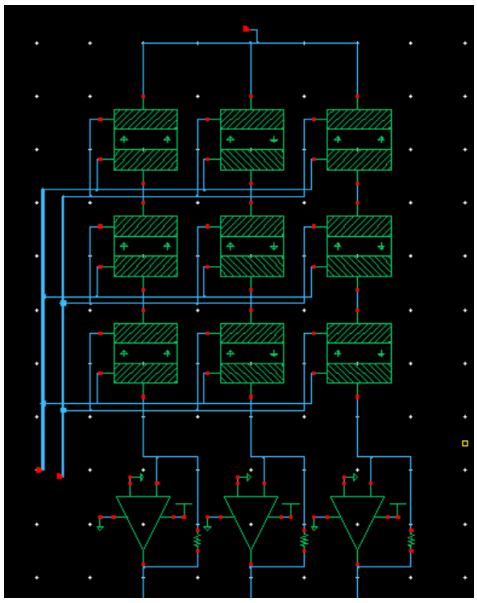

Fig. 5. Schematic of the MRAM crossbar array. Read and write paths are omitted for clarity.

A. Memory Operation

In memory mode, each MTJ cell is accessed through parallel read and write paths as shown in Fig 6. The read operation applies V_{read} to the selected line, and the bitline current indicates whether the MTJ is in the parallel or anti-parallel state. The write operation applies V_{write} with the appropriate polarity to switch the MTJ. Since the complementary MTJ pair stores opposite states, each access returns a differential indication of the stored bit [2].

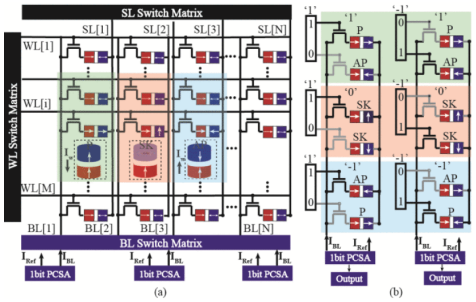

Fig. 6. Reproduced from [2]: (a) Proposed SK-MTJ based pseudo crossbar array architecture; (b) The customized bit-cell design for SK-MTJ. The direction of the write current I_w flowing from PL to the FL is defined as positive.

B. Matrix Multiplication Operation

In computational mode, the MTJ cells in each column are connected in series. The digital input vector drives complementary gate signals to select the appropriate MTJ branch in each cell. The resulting series resistance represents the weighted sum of the digital inputs and stored weights. The column current, measured through Ohm's law, serves as the analog dot-product output.

C. Spectre Waveform Analysis

The simulated Spectre waveform in Fig. 7 shows the output voltage produced by the crossbar column during the analog multiply–accumulate operation. The waveform increases in uniform linear steps as additional MTJ cells are activated. Each step corresponds to the contribution of one cell's resistance to the total series resistance. The consistent step size confirms that the array performs a linear accumulation of weighted inputs, which is the expected behavior for an analog dot-product implemented through resistive summation.

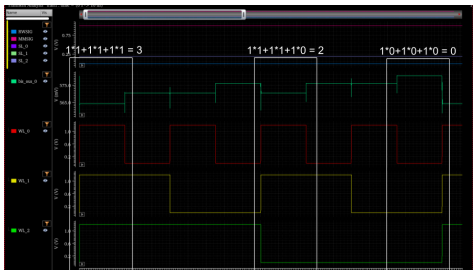

Fig. 7. Spectre simulation waveform of the column output showing linear step increments during the analog multiply–accumulate operation. Here all weights are hard coded to 1 for simplicity.

The Spectre waveform in Fig. 8 shows the bitline response during the MRAM read operation. The waveform exhibits two distinct steady-state levels that correspond to the parallel and anti-parallel states of the MTJ. The stable separation between these levels reflects the difference in resistance between R_{parallel} and $R_{\text{anti-parallel}}$. This behavior confirms correct readout of the stored logic state, since the sensed bitline voltage or current directly represents the MTJ resistance without distortion or drift.

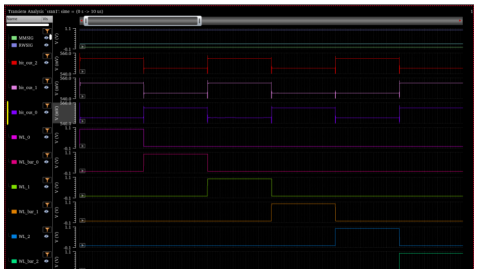

Fig. 8. Spectre simulation waveform of the MRAM read operation showing distinct levels for the parallel and anti-parallel MTJ states.

D. Readout

The electrical outputs of the crossbar array are extracted in the form of currents, which are subsequently converted into measurable voltages through the use of a transimpedance amplifier (TIA). The TIA is implemented using a conventional operational-amplifier topology configured with a fine-tuned feedback resistor. This feedback element establishes the current-to-voltage gain, thereby enabling the translation of the crossbar's current domain response into a corresponding voltage domain signal. The value of the feedback resistor is selected and tuned to ensure that the transimpedance stage generates output voltages within the desired operating range for downstream signal-processing circuitry. The operational amplifier used in the simulation corresponds to a full transistor-level model of the LM741 integrated circuit, as provided by Texas Instruments. The schematic for this is shown in Fig 10.

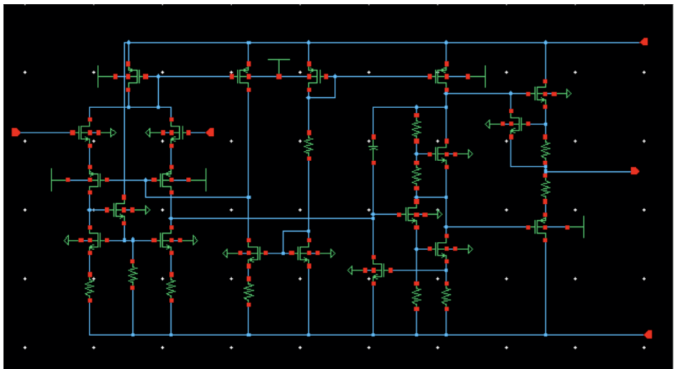

Fig. 9. LM741 transistor model for Op-Amp implementation

Under these conditions, the TIA is designed to produce four distinct, well-separated voltage levels, each of which serves as a unique analog representation of the crossbar's computational state to be subsequently digitized. The TIA is situated at the output of the crossbar array as shown in Fig 11.

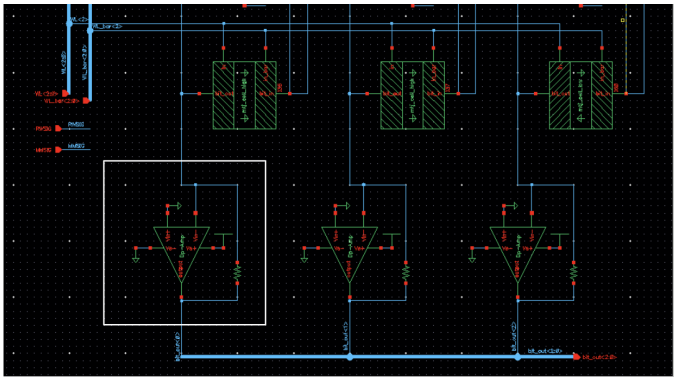

Fig. 10. Op-Amp implementation of TIA using a feedback resistor

E. Analog-Digital Converter

The analog-to-digital conversion stage is realized using a 2-bit flash ADC architecture Fig 12, selected for its simplicity and high conversion speed. This architecture comprises a resistor ladder that intentionally generates a set of non-equally spaced reference voltages, which are routed to a bank of three voltage comparators. These reference voltages are tuned to align with the discrete output levels produced by the TIAs, ensuring that each analog state is accurately captured by the quantization thresholds. The resistor values within the ladder network may be fine-tuned to synthesize arbitrary reference-voltage levels, thereby enabling precise control over the ADC's quantization boundaries. Each comparator is implemented at the transistor level following the circuit topology described in Figure 2a of [3], ensuring consistency with established comparator design methodologies. The comparator waveform is shown in the figure below

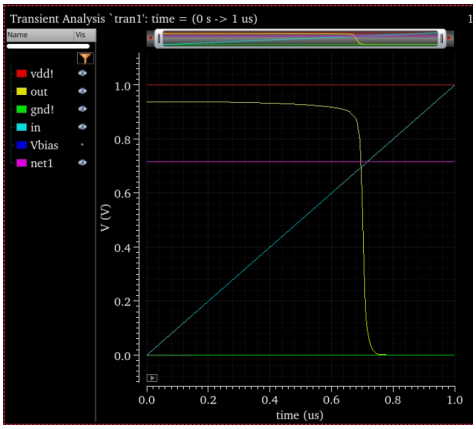

Fig. 11. Waveforms of the comparator showing switching behaviour at the reference voltage (net1) with a ramp input

Given an analog input voltage and the corresponding reference threshold, each comparator outputs a digital high or low level based on its relative magnitude. The resulting outputs are subsequently processed by a set of logic gates that implement a priority-encoding block, which maps the comparator outputs to the final 2-bit digital representation of the TIA-generated signal.

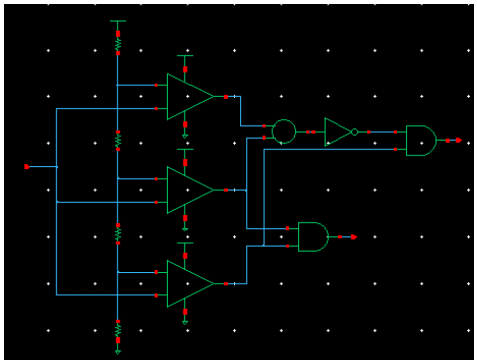

Fig. 12. 2-bit flash ADC architecture that is used including priority encoding

V. Digital Content-Addressable Memory (CAM) Engine

This section presents the design and implementation of the digital CAM engine that interfaces with the analog MRAM array and analog-to-digital conversion circuitry. The objective of the CAM subsystem is to enable fast, word-level associative search across MRAM-stored data while maintaining full synthesizability and compatibility with ASIC design flows. The CAM engine includes four major components: an ADC-to-CAM adapter, a parameterized CAM comparison array, a priority encoder for row selection, and a finite-state search controller.

A. Architecture Overview

The digital CAM follows a structured pipeline. Multi-bit analog readout from the MRAM array is first digitized by an on-chip ADC. The resulting codes for each MTJ cell are processed by a thresholding adapter that produces a binary MTJ state. These binarized rows populate the digital memory image used by the CAM. A dedicated comparison engine evaluates the similarity between a user-supplied search word and each stored row, producing an 8-bit match vector. A priority encoder identifies the lowest-index matching row, and an FSM controller sequences the search across all rows, providing handshake and completion signaling.

The architecture is intentionally modular. Each component synthesizes cleanly to standard cells, enabling standalone verification and hierarchical optimization.

B. ADC-to-CAM Bit-Slicing Adapter

The ADC front-end outputs a 2-bit digital code per MTJ cell. To enable binary CAM comparison, a thresholding operation converts each code into a binary MTJ state.

Let $C_{i,j} \in \{0,1,2,3\}$ denote the ADC output for row i, bit j. The adapter generates a binary state

$$M_{i,j} = \begin{cases} 1, & C_{i,j} \geq T, \\ 0, & C_{i,j} < T, \end{cases} \tag{4}$$

where $T = 1$ by default.

This logic is implemented using a compact 2-bit comparator replicated across all ROWS $\times$ WIDTH MTJ cells, producing a flattened binary vector consumed by the CAM core.

C. CAM Comparison Engine

Given a search word $S \in \{0,1\}^W$ and a stored row $M_i \in \{0,1\}^W$, each bit match is computed using an XNOR operation:

$$b_{i,k} = \neg(M_{i,k} \oplus S_k). \tag{5}$$

Each bitwise comparator is gated by a wordline enable WL. A row match is asserted only if all bits match:

$$\text{row\_match}_i = WL \cdot \bigwedge_{k=0}^{W-1} b_{i,k}. \tag{6}$$

This produces a synthesizable combinational structure based on repeated XNOR–AND cells, analogous to a hardware CAM.

Figure 13 shows the integrated waveform for the search engine, illustrating correct functional interaction between the CAM comparison logic, match-vector generation, and the priority encoder. This waveform provides cycle-by-cycle visibility into how a search word is evaluated across all stored rows.

Several key signals demonstrate proper end-to-end behavior:

- **row_select**: increments through 0–7 as the engine evaluates each stored row.
- **mtj_bits**: displays the 8×3 decoded MTJ pattern used by the CAM during the search.
- **row_match**: asserts only when the current row exactly matches the search word, reflecting the Boolean structure of (6).
- **match_vector**: accumulates row-level matches into an 8-bit vector for priority encoding.
- **match_index**: correctly reports the lowest-index matching row, demonstrating deterministic encoder behavior.
- **valid_match**: asserts when any bit of the match vector is high.

The waveform shows that the search engine correctly identifies matching rows for each programmed search word. For example, when the search word is updated to `5` (binary 101), the CAM comparison block produces a single-row match at row 0, the match vector updates accordingly, and the priority encoder outputs `match_index = 0`. When the search word changes to `7` (111), the match shifts to row 1 as expected. Rows with no match produce a zero match-vector and a deasserted `valid_match`.

Across all tested patterns, the search engine exhibits correct row selection, bitwise comparison, match aggregation, and priority resolution. These results validate the correctness of the CAM datapath and demonstrate that the search engine performs associative search reliably and deterministically.

D. Priority Encoding

The priority encoder resolves multiple simultaneous matches by selecting the lowest-index row. Let $\mathbf{v} =$

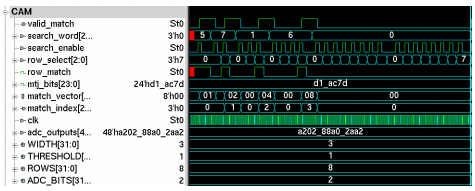

Fig. 13. Waveform of the integrated MTJ search engine showing `row_select`, decoded MTJ bits, row-level match output, match vector, and priority encoder outputs during multi-cycle search.

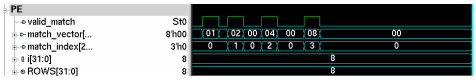

Fig. 14. Waveform of the priority encoder showing the relationship between the input `match_vector`, the asserted `valid_match` flag, and the resulting `match_index`. The encoder reliably selects the lowest-index asserted bit to produce a deterministic row match output.

$[v_0, v_1, \ldots, v_7]$ denote the match vector. The encoder outputs:

$$\text{valid\_match} = \bigvee_{i=0}^{7} v_i, \tag{7}$$

$$\text{match\_index} = \min\{i \mid v_i = 1\}. \tag{8}$$

For an 8-row CAM, a linear scan achieves deterministic behavior and minimal hardware cost.

Figure 14 shows the simulated behavior of the priority encoder, illustrating how the module converts the one-hot row match pattern into a single deterministic row index. The `match_vector` signal presents the eight-bit pattern of matches produced by the comparison engine, and whenever at least one bit is asserted, the encoder raises `valid_match` to indicate that a matching row exists. The encoder then reports the lowest-index asserted bit on `match_index`. This behavior is visible in the waveform: match vectors of `01`, `02`, `04`, and `08` yield output indices of 0, 1, 2, and 3 respectively. When the match vector is zero, `valid_match` deasserts and `match_index` returns to zero. These results confirm deterministic and stable row selection, ensuring correct resolution of CAM search results prior to handoff to the system controller.

E. Finite-State Search Controller

A synchronous FSM manages the search procedure. Its responsibilities include (1) selecting rows sequentially, (2) issuing search strobe pulses, and (3) latching results.

The FSM transitions through the following states:

- **IDLE**: await a `start` signal.
- **SET_ROW**: drive the current row index.
- **PULSE1/PULSE2**: issue comparison strobes.
- **CHECK**: determine whether a match occurred.
- **NEXT_ROW**: advance to the next row if needed.
- **FINISHED**: assert `done` and hold outputs stable.

This FSM ensures deterministic latency and clean separation between control and datapath logic.

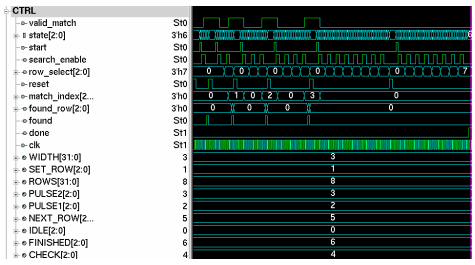

Fig. 15. Waveform of the search controller FSM showing transitions through the IDLE, SET_ROW, PULSE1, PULSE2, CHECK, NEXT_ROW, and FINISHED states. The signals `row_select`, `match_index`, `found`, and `done` demonstrate correct sequencing and deterministic control of the CAM search procedure.

Figure 15 illustrates the cycle-by-cycle behavior of the search controller FSM, showing how it sequences row evaluation, strobe generation, and match detection. Once the `start` signal is asserted, the FSM exits the **IDLE** state and enters **SET_ROW**, where the current row index is driven onto `row_select`. The subsequent **PULSE1** and **PULSE2** states generate the comparison strobes required by the CAM comparison engine. After pulsing, the controller transitions to **CHECK**, during which the incoming `valid_match` and `match_index` signals are sampled. If no match is found, the FSM advances to **NEXT_ROW**, increments the row counter, and repeats the procedure. This progression is clearly visible in the waveform: `row_select` increments monotonically from 0 through 7 as the search iterates across all rows. When a match occurs (e.g., rows 1, 2, or 3 in the example), the `found` flag momentarily asserts and the matched row is latched into `found_row`. Once all rows have been processed or a terminating match is found, the FSM enters the **FINISHED** state, asserting `done` and holding outputs stable until the next search command. This behavior verifies that the FSM correctly orchestrates the CAM search pipeline and enforces deterministic ordering of comparison and capture events.

F. Simulation and Verification

The behavior of the complete MRAM search system was verified using a SystemVerilog testbench that exercises the ADC decoding path, CAM comparison logic, priority encoding, and the search controller FSM. The stored MTJ data were initialized to eight distinct three-bit patterns:

$$R_0 = 101, \quad R_1 = 111, \quad R_2 = 001, \quad R_3 = 110,$$
$$R_4 = 010, \quad R_5 = 011, \quad R_6 = 100, \quad R_7 = 110.$$

A set of representative search outcomes is summarized in Table I. These results demonstrate correct positive matches, correct handling of duplicate patterns, and expected behavior for search words that do not appear in any stored row.

TABLE I
REPRESENTATIVE CAM SEARCH RESULTS

Search Word	Match	Row	Notes
101	Yes	0	Correct positive match
111	Yes	1	Correct positive match
001	Yes	2	Correct positive match
110	Yes	3	First of two identical rows
000	No	–	Correct negative match

Figure 16 shows a representative simulation waveform of the integrated system in operation. For each applied search word, the ADC-coded MTJ outputs are decoded into binary values and passed to the CAM comparison array. The controller then steps through each stored row in sequence, issuing comparison strobes and monitoring the resulting match signals. When a row matches the search pattern, the system asserts a match indication and latches the corresponding index into the `found_row` register. This is evident in the waveform, where searches for `101`, `111`, and `001` correctly produce results of rows 0, 1, and 2. In contrast, search patterns not present in memory result in no match assertion throughout the scan.

Across all trials, the `done` signal reliably marks search completion, whether or not a valid match is found. The controller terminates deterministically after evaluating all rows, and all system outputs stabilize as expected. These waveform results confirm that the full mixed-signal search architecture performs correct pattern decoding, row-by-row comparison, priority resolution, and orderly search completion, demonstrating robust and consistent end-to-end operation.

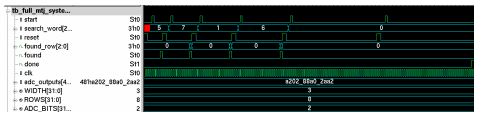

Fig. 16. Full-system simulation waveform showing correct decoding of search words, sequential row evaluation, match detection, and deterministic search completion. The signals `found`, `found_row`, and `done` confirm proper end-to-end operation.

VI. METHODS

Both analog and digital components of the system were designed and verified using industry standard electronic design automation tools. Analog circuit design was performed using Virtuoso Schematic Editor, with transient, DC, and AC simulations carried out using the Spectre circuit simulator. These tools were used to characterize device level behavior, verify biasing conditions, and evaluate the interaction between magnetic tunnel junction elements and peripheral CMOS circuitry.

Digital control and data path logic were described in SystemVerilog and verified using the VCS simulation environment. Functional verification focused on correctness of state machines, timing of control signals, and proper sequencing of read and write operations. Following functional verification, logic synthesis was performed using Design Vision to map the SystemVerilog descriptions onto a standard cell library, enabling area, timing, and power estimation at the gate level.

Physical implementation of the synthesized digital design was performed using Innovus for automated place and route. Floorplanning, power grid generation, standard cell placement, clock tree synthesis, and routing were completed to produce layout level representations suitable for timing closure analysis. Post layout timing verification was carried out using extracted parasitic information to assess the impact of interconnect resistance and capacitance.

An effort was made to generate a gate level netlist after synthesis and re import this netlist into the schematic environment to perform full mixed signal simulations using Spectre. The goal of this step was to achieve a unified analog digital co simulation framework capable of capturing both device level analog effects and system level digital behavior within a single simulation environment. Although partial progress was achieved, tool flow integration challenges prevented completion of this mixed domain simulation. As a result, full analog digital co simulation was not finalized and is left outside the scope of this work.

A. Physical Design, Area, and Timing Results

The digital CAM subsystem and search controller were synthesized, placed, and routed using a 45 nm standard-cell library to evaluate physical design feasibility and quantify implementation cost. Both unoptimized and optimized synthesis runs were performed to assess the benefits of logic restructuring and buffer reduction.

Table II summarizes the area-related results before and after optimization. The unoptimized design contains 300 standard cells and occupies approximately $712\,\mu\text{m}^2$. After optimization, the cell count decreases slightly to 296 cells, with a corresponding reduction in total area to $707\,\mu\text{m}^2$. Although the improvements are modest, they reflect consistent reductions in combinational logic and buffer usage. These results confirm that the CAM search engine and controller occupy only a small silicon footprint, making them suitable for integration alongside lightweight mixed-signal peripheral circuitry.

TABLE II
AREA SUMMARY BEFORE AND AFTER LOGIC OPTIMIZATION

Metric	Unoptimized	Optimized	Change
Total Cells	300	296	-1.3%
Combinational Cells	238	234	-1.7%
Sequential Cells	20	20	—
Buffer / Inverter Count	129	125	-3.1%
Total Area (μm^2)	712.40	707.24	-0.7%

Static timing analysis prior to place-and-route indicates a worst-case critical-path delay of 2.57 ns from the row-selection register through the CAM comparison logic to the match-vector register. The design meets timing with over 1 ns of positive slack, demonstrating that the CAM can support multi-hundred-MHz operation in pre-layout conditions.

After automated place-and-route, extracted parasitics were incorporated for post-layout timing analysis. As shown in Table III, the representative unconstrained critical path increases to approximately 2.67 ns due to realistic interconnect loading and routing complexity. Despite this increase, the synchronous

logic exhibits no setup or hold violations, confirming that the design remains timing-clean after physical implementation.

TABLE III
TIMING SUMMARY BEFORE AND AFTER APR

Metric	Pre-APR	Post-APR
Critical Path Delay (ns)	2.57	2.67
Slack	+1.11	+1.02
Timing Violations	None	None

Figure 17 shows the final place-and-route layout of the synthesized CAM subsystem and controller. The design routes cleanly with no congestion or DRC violations, and the compact footprint highlights the feasibility of embedding the digital search engine adjacent to the MRAM crossbar within a mixed-signal compute macro. These physical design results demonstrate that the system is not only functionally correct but also physically implementable using standard ASIC design flows.

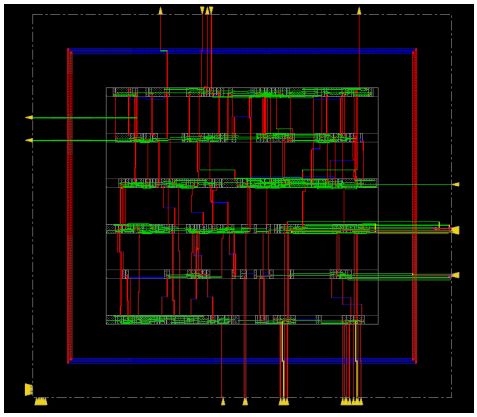

Fig. 17. Final place-and-route layout of the synthesized digital CAM subsystem and search controller.

VII. CONCLUSION

This work demonstrates a complete MRAM-based in-memory computing macro that unifies storage, analog computation, and digital associative search within a single architecture. We designed and validated an MTJ crossbar array capable of operating in both memory and compute modes, and we implemented an analog processing chain that includes the readout network, transimpedance amplifier, and flash ADC required to convert MTJ-derived signals into reliable digital outputs. In parallel, we developed a synthesizable digital CAM engine that performs word-level associative lookup and integrates cleanly with the analog front end. The entire system was constructed to support a standard Verilog design and verification flow, allowing mixed-signal MRAM behavior to be evaluated within a conventional digital development environment. The resulting macro shows how nonvolatile MTJ devices can serve both as memory elements and as active analog compute units, providing a practical pathway toward scalable in-memory accelerators that reduce data movement and overcome the limitations imposed by the memory wall.

As the project progressed, we discovered that FreePDK45 was not suitable for completing a full mixed-signal integration of our system. Although the PDK supports standard digital synthesis and physical design, and also provides the environment needed for analog development in Virtuoso, it does not offer a mechanism to combine placed and routed digital blocks with Spectre-based analog circuits inside a single mixed-signal hierarchy. Because of this constraint, the analog components of the MRAM compute path, including the MTJ crossbar, TIA, and ADC, were implemented and verified entirely within Virtuoso, while the CAM engine and control logic were synthesized, placed, routed, and validated in the digital APR flow. Both domains operate correctly on their own, but they cannot be brought together into a unified top-level layout within FreePDK45. Producing a complete mixed-signal IC would require migrating the design to a PDK that natively supports RTL-to-GDS mixed-signal integration, such as FreePDK45 or an industry-grade process. For this reason, the report focuses on the successful design and testing of the individual analog and digital subsystems, along with the interface specifications required for future integration.

VIII. CHALLENGES AND FUTURE WORK

Several technical challenges were encountered throughout the design and simulation workflow. A primary difficulty involved accurately representing the MTJ device within the Cadence environment. Due to the lack of an accessible open source PDK with native MTJ support, we employed a simplified resistive model to approximate the circuit-level behavior of the MTJ. While this abstraction enabled functional validation of the overall architecture, it inevitably limited the device-level fidelity of the simulations. Another significant challenge arose in the analog-to-digital conversion subsystem. The transistor-level LM741 operational amplifier model demonstrated unreliable behavior when configured as a high-gain open-loop comparator, making it unsuitable for the flash ADC architecture. As a result, we implemented custom comparator circuits based on the topology outlined in [3], ensuring stable operation and consistent thresholding behavior at the required voltage levels. A further obstacle involved the generation of the map file necessary for exporting the digital CAM search engine as a GDSII layout. In the absence of a properly formed map file, the digital CAM could not be imported into Cadence Virtuoso, preventing full-system physical integration and verification of interconnections between the crossbar array and the digital search engine.

Future work will concentrate on a limited set of targeted enhancements intended to refine and extend the capabilities of the current system. One planned development is the implementation of an FSM that will serve as a centralized control unit for the crossbar array, governing readout sequencing, operational coordination, and mode selection with greater architectural coherence. Another focused effort will involve the design and calibration of a dedicated 2-bit ADC specifically optimized for the voltage ranges associated with matrix–vector multiplication, thereby complementing the existing ADC utilized

for CAM-based operations. The FSM will additionally manage the multiplexing framework required to seamlessly route signals between these two ADC pathways depending on the computational mode in use. Finally, a remaining objective is the generation of a complete GDSII layout for the digital CAM search engine, enabling its import into Cadence Virtuoso and permitting full physical integration of the crossbar array with the digital CAM for comprehensive layout-level verification.

REFERENCES

[1] S. Jung, H. Lee, S. Myung *et al.*, "A crossbar array of magnetoresistive memory devices for in-memory computing," *Nature*, vol. 601, no. 7892, pp. 211–216, 2022. [Online]. Available: https://doi.org/10.1038/s41586-021-04196-6

[2] B. Pan, D. Zhang, X. Zhang, H. Wang, J. Bai, J. Yang, Y. Zhang, W. Kang, and W. Zhao, "Skyrmion-induced memristive magnetic tunnel junction for ternary neural network," *IEEE Journal of the Electron Devices Society*, vol. 7, pp. 529–533, 2019.

[3] W. Jendernalik, "An ultra-low-energy analog comparator for a/d converters in cmos image sensors," *Circuits, Systems, and Signal Processing*, vol. 36, no. 12, pp. 4829–4843, Aug. 2017.